Análisis de Impacto de Cambios de Dependencias en Proyectos Kotlin Multiplatform: Pipeline No Invasivo que Integra Análisis Estático (KSP, Detekt), Detección de Regresión de Pantallas (DroidBot) y Visualización 3D (CodeCharta Extendido)

Esteban Castelblanco Gómez

Universidad de Los Andes

Departamento de Ingeniería de Sistemas y Computación

Bogotá, Colombia

2026

Análisis de Impacto de Cambios de Dependencias en Proyectos Kotlin Multiplatform: Pipeline No Invasivo que Integra Análisis Estático (KSP, Detekt), Detección de Regresión de Pantallas (DroidBot) y Visualización 3D (CodeCharta Extendido).

Esteban Castelblanco Gómez

Universidad de Los Andes

Presentado en cumplimiento parcial de los requisitos para el grado de:

Ingeniero de Sistemas y Computación

Director:

Prof. Camilo Andrés Escobar Velásquez

Universidad de Los Andes

Departamento de Ingeniería de Sistemas y Computación

Bogotá, Colombia

2026

Resumen

Este documento presenta el diseño, implementación y validación de un pipeline automatizado y no invasivo para el **análisis de impacto de cambios de dependencias** en proyectos Kotlin Multiplatform (KMP). Cuando una dependencia de un proyecto KMP se actualiza —ya sea por actualización de versión, migración de librería, o alerta de Dependabot— el pipeline responde dos preguntas críticas: (1) ¿qué módulos y clases del código se ven afectados por el cambio? y (2) ¿qué pantallas de la aplicación se rompen o se ven impactadas?

Para responder la primera pregunta, el pipeline emplea análisis estático mediante KSP (grafo de dependencias vía AST), Detekt (complejidad ciclomática) y Kover (cobertura de pruebas), construyendo un mapa de propagación de impacto desde la dependencia modificada hasta las clases y módulos que la consumen. Para responder la segunda, integra DroidBot y AutoDroid, herramientas de exploración automática de UI sin instrumentación, que ejecutan la aplicación antes y después del cambio de dependencia para detectar pantallas que fallan, cambian de comportamiento, o dejan de ser alcanzables.

Los resultados se consolidan y visualizan en una extensión de CodeCharta (*open-source*), donde el modo Delta permite comparar visualmente el estado del proyecto antes y después del cambio de dependencia: los edificios (clases) afectados se resaltan, y las pantallas impactadas se conectan con las clases modificadas. El principio de diseño central es el **Shadowing**: el pipeline opera sobre copias temporales del repositorio sin modificar los archivos de configuración del equipo de desarrollo.

Palabras clave: Kotlin Multiplatform, análisis de impacto, cambio de dependencias, Dependabot, análisis estático, análisis dinámico, DroidBot, CodeCharta, metáfora de ciudad, grafo de dependencias, KSP, regresión de UI, pipeline no invasivo.

Índice

Resumen **1**

1. Introducción **5**

1.1. Contexto 5

1.2. Problemática 6

1.3. Justificación 7

1.4. Objetivos 7

1.4.1. Objetivo General 7

1.4.2. Objetivos Específicos 8

1.4.3. Pregunta de Investigación 8

2. Marco Teórico **9**

2.1. Marco Conceptual 9

2.1.1. Kotlin Multiplatform (KMP) 9

2.1.2. Mecanismo `expect/actual` 9

2.1.3. Complejidad Ciclomática y Análisis Estático 10

2.1.4. Cobertura de Pruebas en KMP 10

2.1.5. Procesamiento de Símbolos (KSP) 10

2.1.6. Generación Automática de Inputs 10

2.1.7. Visualización de Software con Metáfora de Ciudad 11

2.1.8. Análisis de Impacto de Cambios y Gestión de Dependencias 11

2.1.9. Gradle Init Scripts y el Principio de *Shadowing* 11

2.2. Estado del Arte 11

3. Metodología **14**

3.1. Fase 1: Shadowing y Preparación de Versiones (OE1) 15

3.2. Fase 2: Análisis Estático de Propagación de Impacto (OE1, OE2) 15

3.3. Fase 3: Análisis Dinámico de Regresión de Pantallas (OE3) 15

3.4. Fase 4: Consolidación y Cruce Código–Pantalla (OE4) 15

3.5. Fase 5: Visualización en CodeCharta Extendido (OE4) 16

3.6. Fase 6: Evaluación de Exactitud del Pipeline (OE5) 16

3.6.1. Evaluación del Análisis Estático: ¿Detecta correctamente las clases afectadas? 17

3.6.2. Evaluación del Análisis Dinámico: ¿Detecta correctamente las pantallas rotas? 17

3.6.3. Evaluación de la Visualización: ¿La extensión de CodeCharta conecta las dimensiones? 17

4. Resultados Esperados **19**

5. Conclusiones y Trabajos Futuros **20**

Índice de figuras

1. Pipeline de análisis de impacto de cambio de dependencias en proyectos KMP. 14

Índice de cuadros

1. Resumen del Estado del Arte por eje temático. 13
2. Mapeo de métricas a dimensiones visuales en CodeCharta extendido. . . . 16
3. Métricas de evaluación por componente del pipeline. 20

1. Introducción

1.1. Contexto

El mantenimiento de dependencias externas es una de las actividades más críticas y frecuentes en el ciclo de vida del software moderno Janakaraj (2025). Herramientas como Dependabot automatizan la detección de actualizaciones disponibles y vulnerabilidades de seguridad en dependencias, pero no informan sobre el **impacto** que un cambio de dependencia tiene sobre el código que la consume ni sobre las pantallas de la aplicación que podrían verse afectadas.

Kotlin Multiplatform (KMP) agrava este desafío. KMP permite compartir lógica de negocio entre Android e iOS mientras preserva la experiencia de usuario nativa de cada plataforma Janakaraj (2025), a diferencia de Flutter o React Native que abstraen la UI completa Suri et al. (2023). Sin embargo, esta arquitectura introduce una complejidad particular en el manejo de dependencias: una dependencia compartida en `commonMain` (como Ktor para networking o SQLDelight para persistencia) puede impactar simultáneamente las pantallas de Android *y* de iOS, y el mecanismo `expect/actual` crea fronteras de abstracción que dificultan rastrear la propagación del impacto.

A escala industrial, este problema se magnifica. Beriukhov Beriukhov (2025) documenta un proyecto con más de 1,000 módulos Gradle donde la migración de la capa de red (cambio de dependencia de la infraestructura existente a Ktor) resultó ser el desafío más complejo, precisamente porque la propagación del impacto era difícil de predecir sin herramientas adecuadas.

La visualización de software ofrece un camino prometedor para comunicar el impacto de estos cambios. La metáfora de ciudad 3D, validada empíricamente por Wettel et al. Wettel et al. (2011) (+24 % correctitud, -12 % tiempo en tareas de comprensión), y su implementación *open-source* en CodeCharta MaibornWolff GmbH (2024), permiten representar sistemas complejos de forma intuitiva Moreno-Lumbreras et al. (2024); Jeffery (2023). El modo Delta de CodeCharta puede comparar dos versiones de un proyecto, pero actualmente no incorpora información de **qué pantallas** de la aplicación se ven afectadas por los cambios detectados.

Por otro lado, herramientas de análisis dinámico como DroidBot Li et al. (2017) pueden explorar automáticamente una app y generar un grafo de transición de UI (UTG) que mapea qué pantallas son alcanzables y qué métodos se ejecutan en cada una. Al ejecutar DroidBot *antes* y *después* de un cambio de dependencia, es posible detectar pantallas que

fallan, cambian de comportamiento, o dejan de ser alcanzables.

Sin embargo, no existe un pipeline que integre ambas perspectivas —el análisis estático de propagación de impacto en código y el análisis dinámico de regresión en pantallas— para responder de forma automática y visual: *si cambio esta dependencia, ¿qué código se afecta y qué pantallas se rompen?*

1.2. Problemática

Cuando una dependencia cambia en un proyecto KMP, los equipos de desarrollo enfrentan tres problemas concretos que las herramientas actuales no resuelven de forma integrada:

1. **Propagación invisible del impacto en código.** Una dependencia en `commonMain` (por ejemplo, Ktor 2.x → 3.x) puede afectar clases en `commonMain`, `androidMain` e `iosMain` a través de cadenas de dependencias transitivas y fronteras `expect/actual`. Las herramientas como Dependabot alertan sobre la disponibilidad de la actualización, pero no informan *qué clases* del proyecto consumen la dependencia modificada ni cómo se propaga el impacto por el grafo de dependencias internas Janakaraj (2025).

2. **Regresión de pantallas no detectada.** Un cambio en una dependencia de red o de persistencia puede causar que pantallas de la aplicación fallen, muestren datos incorrectos, o dejen de ser alcanzables. Las herramientas de análisis estático no pueden detectar estas regresiones de UI porque requieren ejecutar la aplicación. DroidBot Li et al. (2017) puede explorar pantallas automáticamente, pero no está diseñado para comparar el estado de las pantallas *antes* y *después* de un cambio de dependencia.

3. **Falta de visualización del impacto.** Incluso si se detectan las clases afectadas y las pantallas rotas, no existe una forma visual de comunicar este impacto. CodeCharta MaibornWolff GmbH (2024) ofrece un modo Delta para comparar dos versiones del código, pero no incorpora la dimensión de *qué pantallas* de la app corresponden a las clases modificadas. La conexión código ↔ pantalla es invisible Moreno-Lumbreras et al. (2024).

Beriukhov Beriukhov (2025) ilustra este problema en la práctica: al migrar la capa de red de un proyecto con 1,000+ módulos, los contratos diferían entre plataformas, había *features* muertas con *headers* no utilizados, y problemas de compatibilidad con Ktor y QUIC que solo se descubrieron durante la integración. Un pipeline de análisis de impacto habría anticipado estos problemas.

1.3. Justificación

La justificación de este trabajo se sustenta en tres ejes:

Necesidad industrial demostrada. Beriukhov Beriukhov (2025) documenta que en proyectos KMP a gran escala (1,000+ módulos, 150+ desarrolladores), la migración de dependencias core (como la capa de red) es el desafío más complejo, precisamente por la dificultad de predecir el impacto. La distinción entre migrar una librería aislada versus un *feature* de producto con decenas de dependencias *core* subraya la necesidad de un análisis de impacto automatizado Beriukhov (2025).

Herramientas existentes pero desconectadas. Las piezas del rompecabezas existen: KSP Google (2024) puede construir grafos de dependencias, DroidBot Li et al. (2017) puede explorar pantallas sin instrumentación, y CodeCharta MaibornWolff GmbH (2024) puede visualizar diferencias entre versiones. Sin embargo, ninguna las conecta en un flujo integrado que responda "si cambio esta dependencia, ¿qué código y qué pantallas se afectan?". Además, CodeCharta es *open-source*, lo que permite extenderlo para incorporar la dimensión de impacto en pantallas.

Base empírica de la visualización. La metáfora de ciudad 3D está validada por Wettel et al. ($F(1, 37) = 14{,}722$, $p = {,}001$) Wettel et al. (2011), con la ventaja creciendo en sistemas grandes (+29.62 %). Balogh et al. Balogh et al. (2016) extendieron la metáfora para incluir métricas de *testing*, y Moreno-Lumbreras et al. Moreno-Lumbreras et al. (2024) confirmaron su vigencia. La extensión propuesta —añadir impacto de dependencias y regresión de pantallas— es una contribución natural a esta línea de investigación Custura and Vescan (2025).

1.4. Objetivos

1.4.1. Objetivo General

Diseñar e implementar un pipeline automatizado y no invasivo que, ante un cambio de dependencia en un proyecto Kotlin Multiplatform, identifique automáticamente (1) qué módulos y clases del código se ven afectados mediante análisis estático del grafo de dependencias, (2) qué pantallas de la aplicación se rompen o cambian de comportamiento mediante análisis dinámico con DroidBot, y (3) visualice el impacto de forma integrada en una extensión de CodeCharta.

1.4.2. Objetivos Específicos

OE1. Desarrollar un Init Script de Gradle Gradle Inc. (2024) que inyecte un procesador KSP personalizado y Detekt en un proyecto KMP sin modificar sus archivos de *build*, implementando el principio de *Shadowing*.

OE2. Diseñar un procesador KSP Google (2024) que, dada una dependencia específica, genere el **grafo de propagación de impacto**: el árbol de clases y funciones que consumen directa o transitivamente esa dependencia, incluyendo las fronteras `expect/actual` entre módulos.

OE3. Integrar DroidBot Li et al. (2017) para ejecutar la aplicación Android *antes* y *después* del cambio de dependencia, comparando los UTG (*UI Transition Graphs*) resultantes para detectar pantallas con regresión: pantallas que fallan, cambian de comportamiento, o dejan de ser alcanzables.

OE4. Extender CodeCharta MaibornWolff GmbH (2024) (*open-source*) para incorporar una nueva dimensión de impacto: conectar las clases afectadas (del grafo estático) con las pantallas impactadas (del análisis dinámico), visualizándolas como edificios resaltados con conexiones a sus pantallas correspondientes.

OE5. Evaluar la exactitud del pipeline mediante métricas de *Information Retrieval* (Precision, Recall, F1-Score) sobre al menos dos escenarios de cambio de dependencia real en proyectos KMP, comparando los resultados automáticos contra una verificación manual experta tanto para las clases afectadas (análisis estático) como para las pantallas impactadas (análisis dinámico).

1.4.3. Pregunta de Investigación

¿Con qué nivel de precisión y cobertura puede un pipeline no invasivo, que combina análisis estático del grafo de dependencias y análisis dinámico de regresión de pantallas, identificar correctamente las clases afectadas y las pantallas impactadas ante un cambio de dependencia en un proyecto Kotlin Multiplatform?

2. Marco Teórico

2.1. Marco Conceptual

2.1.1. Kotlin Multiplatform (KMP)

Kotlin Multiplatform Mobile es una tecnología desarrollada por JetBrains que permite compartir código Kotlin entre Android e iOS JetBrains (2025). Su diferenciador clave frente a Flutter o React Native es que comparte únicamente la lógica de negocio, preservando la UI nativa de cada plataforma Janakaraj (2025). La arquitectura de un proyecto KMP se organiza en tres capas:

- `commonMain`: Módulo compartido con lógica de negocio, modelos de datos, capa de red, manejo de estado y funciones de utilidad. Alto potencial de compartición.
- `androidMain`: Implementaciones nativas de Android usando Kotlin y el *toolkit* nativo. Bajo potencial de compartición.
- `iosMain`: Implementaciones nativas de iOS que interactúan con UIKit o SwiftUI. Bajo potencial de compartición.

2.1.2. Mecanismo `expect/actual`

El puente entre código compartido y nativo en KMP es el mecanismo `expect/actual` JetBrains (2025). Una declaración `expect` en el módulo común define un contrato, mientras que la implementación `actual` en cada módulo de plataforma proporciona la realización concreta. El compilador de Kotlin garantiza seguridad de tipos en tiempo de compilación.

Listing 1: Ejemplo del patrón `expect/actual` para un driver de base de datos.

```kotlin
// commonMain/kotlin/data/DatabaseDriverFactory.kt
expect class DatabaseDriverFactory {
    fun createDriver(): SqlDriver
}

// androidMain/kotlin/data/DatabaseDriverFactory.android.kt
actual class DatabaseDriverFactory(private val context: Context) {
    actual fun createDriver(): SqlDriver {
        return AndroidSqliteDriver(AppDatabase.Schema, context, "
            AppDatabase.db")
    }
}

```

```kotlin
// iosMain/kotlin/data/DatabaseDriverFactory.ios.kt
actual class DatabaseDriverFactory {
    actual fun createDriver(): SqlDriver {
        return NativeSqliteDriver(AppDatabase.Schema, "AppDatabase.db")
    }
}
```

2.1.3. Complejidad Ciclomática y Análisis Estático

La complejidad ciclomática, propuesta por McCabe McCabe (1976), cuantifica el número de caminos linealmente independientes a través del grafo de control de flujo de un programa. Es ampliamente utilizada como indicador de mantenibilidad y riesgo de errores. Detekt Detekt Team (2024) es la herramienta estándar de análisis estático para Kotlin, capaz de detectar *code smells* y calcular complejidad ciclomática. Novendra y Sunindyo Novendra and Sunindyo (2024) propusieron recientemente un detector de *bad smells* específicos de Kotlin para aplicaciones, logrando un F-measure de 100 % para detección de *Brain Class* y 93.77 % para *God Class*.

2.1.4. Cobertura de Pruebas en KMP

Kover JetBrains (2024) es la herramienta oficial de JetBrains para medir cobertura de código en proyectos Kotlin, incluyendo KMP. Janakaraj identifica tres capas de *testing* en proyectos KMP: *tests* del módulo común, *tests* específicos de plataforma, y *tests* de integración Janakaraj (2025).

2.1.5. Procesamiento de Símbolos (KSP)

Kotlin Symbol Processing (KSP) Google (2024) es una API de Google para construir procesadores ligeros que operan sobre el Árbol de Sintaxis Abstracta (AST) de programas Kotlin. A diferencia de `kapt` (basado en anotaciones Java), KSP es nativo de Kotlin y soporta *targets* multiplataforma. En este trabajo, se utiliza KSP para generar un grafo de dependencias de alta fidelidad que incluya las fronteras `expect/actual`.

2.1.6. Generación Automática de Inputs

DroidBot Li et al. (2017) es un generador ligero de *inputs* de prueba que no requiere instrumentación. Construye un *UI Transition Graph* (UTG) en tiempo real mediante una estrategia *depth-first* y utiliza utilidades nativas (ADB, UI Automator). AutoDroid Wen et al. (2024) extiende DroidBot con LLMs (GPT-3.5-turbo) para exploración orientada

a tareas. LLMDroid Wang et al. (2025) propuso un framework para mejorar la cobertura de herramientas de testing existentes mediante guía de LLMs, reportando mejoras significativas en cobertura de código y de actividades.

2.1.7. Visualización de Software con Metáfora de Ciudad

La metáfora de ciudad 3D, propuesta por Wettel y Lanza Wettel et al. (2011), representa clases como edificios y paquetes como distritos, mapeando métricas de software sobre propiedades visuales. Moreno-Lumbreras et al. Moreno-Lumbreras et al. (2024) confirmaron la influencia activa y creciente de esta metáfora en la investigación de visualización de software. CodeCharta MaibornWolff GmbH (2024) es una implementación *open-source* que soporta múltiples fuentes de métricas y ofrece un modo Delta para comparar versiones.

2.1.8. Análisis de Impacto de Cambios y Gestión de Dependencias

El análisis de impacto de cambios (*change impact analysis*) busca determinar qué partes de un sistema se ven afectadas por una modificación Janakaraj (2025); Beriukhov (2025). En el contexto de dependencias externas, herramientas como Dependabot automatizan la detección de actualizaciones y vulnerabilidades, pero operan a nivel de artefacto ("la versión X de la librería Y tiene una vulnerabilidad") sin informar sobre la propagación del impacto dentro del código del proyecto. En proyectos KMP, una dependencia compartida en `commonMain` puede propagarse a través de cadenas de `import`, fronteras `expect/actual`, y capas arquitectónicas (red, persistencia, UI), haciendo que el análisis de impacto requiera un grafo de dependencias a nivel de clases y funciones, no solo de artefactos.

2.1.9. Gradle Init Scripts y el Principio de *Shadowing*

Los Init Scripts de Gradle Gradle Inc. (2024) permiten inyectar configuración en un proyecto sin modificar sus archivos de *build*. Se ejecutan antes de cualquier evaluación de *build scripts*. En este trabajo, se utiliza un Init Script para implementar el principio de *Shadowing*: clonar el repositorio, inyectar las herramientas de análisis, y operar sobre la copia sin alterar el repositorio original.

2.2. Estado del Arte

El estado del arte se organiza en cuatro ejes temáticos que convergen en la propuesta de este trabajo.

Eje 1: Visualización de software con metáfora de ciudad. El trabajo de Wettel et al.

Wettel et al. (2011) estableció la base empírica con un experimento controlado (41 participantes, +24 % correctitud, -12 % tiempo). Balogh et al. Balogh et al. (2016) extendieron la metáfora para incluir métricas de *testing*. Jeffery Jeffery (2023) realizó un *survey* comprensivo de las implementaciones existentes. Moreno-Lumbreras et al. Moreno-Lumbreras et al. (2024) publicaron en 2024 una revisión sistemática demostrando la presencia activa y creciente de la metáfora de ciudad en la investigación. Custura y Vescan Custura and Vescan (2025) validaron en 2025 una metáfora alternativa (ropa) con 41 sujetos, confirmando el interés vigente en metáforas visuales para comprensión de software.

Eje 2: Kotlin Multiplatform y arquitecturas multiplataforma. Janakaraj Janakaraj (2025) sintetizó la arquitectura de KMP, incluyendo `expect/actual`, estrategias de repositorio (monorepo vs. multi-repo), y consideraciones de implementación (inyección de dependencias, manejo de estado, tres capas de testing). Beriukhov Beriukhov (2025) reportó la experiencia industrial con 1,000+ módulos y un A/B test con resultados neutrales de rendimiento. Suri et al. Suri et al. (2023) realizaron un análisis empírico comparativo de Kotlin, React Native y Flutter. Majchrzak et al. Majchrzak et al. (2017) y Biørn-Hansen et al. Biørn-Hansen et al. (2017) abordaron la calidad y las tendencias en desarrollo multiplataforma.

Eje 3: Análisis dinámico y testing automatizado. Pan et al. Pan et al. (2020) propusieron testing basado en *reinforcement learning* con exploración dirigida por curiosidad. Li et al. Li et al. (2017) presentaron DroidBot como generador ligero de inputs sin instrumentación. Hao et al. Hao et al. (2014) propusieron PUMA como framework programable para análisis dinámico a gran escala. Wen et al. Wen et al. (2024) extendieron DroidBot con LLMs (AutoDroid). Wang et al. Wang et al. (2025) demostraron con LLMDroid que la guía de LLMs mejora significativamente la cobertura de herramientas de testing existentes.

Eje 4: Análisis estático y métricas de calidad. McCabe McCabe (1976) formalizó la complejidad ciclomática como métrica fundamental de mantenibilidad. Merino et al. Merino et al. (2018) realizaron una revisión sistemática de la evaluación de herramientas de visualización de software. Novendra y Sunindyo Novendra and Sunindyo (2024) propusieron un detector de *bad smells* específicos de Kotlin con F-measure superior al 85 %.

Cuadro 1: Resumen del Estado del Arte por eje temático.

Referencia	**Enfoque**	**Aporte clave**
Wettel et al. (2011)	Visualización 3D	+24 % correctitud, -12 % tiempo
Moreno-Lumbreras et al. (2024)	*Survey*	Metáfora de ciudad activa y creciente
Janakaraj (2025)	Arquitectura KMP	`expect/actual`, DI, *state mgmt*
Beriukhov (2025)	KMP a escala	1,000+ módulos, A/B test neutral
Li et al. (2017)	DroidBot	UI sin instrumentación, UTG
Wang et al. (2025)	LLMDroid	Mejora de cobertura con guía de LLMs
McCabe (1976)	Complejidad	Métrica de mantenibilidad

Brecha identificada. Si bien existen contribuciones sólidas en (i) visualización 3D de software, (ii) arquitectura KMP y gestión de dependencias, (iii) análisis dinámico de UI con LLMs, y (iv) métricas de calidad estática, no existe un pipeline que las integre para responder una pregunta específica: *cuando cambia una dependencia en un proyecto KMP, ¿qué código se afecta y qué pantallas se rompen?* Tampoco existe una extensión de CodeCharta que conecte visualmente el impacto en código con el impacto en pantallas. Este trabajo se posiciona para cubrir ambas brechas.

3. Metodología

La metodología se estructura en cinco fases que operan sobre dos versiones del proyecto: `ANTES` (con la dependencia original) y `DESPUES` (con la dependencia actualizada). La Figura 1 ilustra el flujo general.

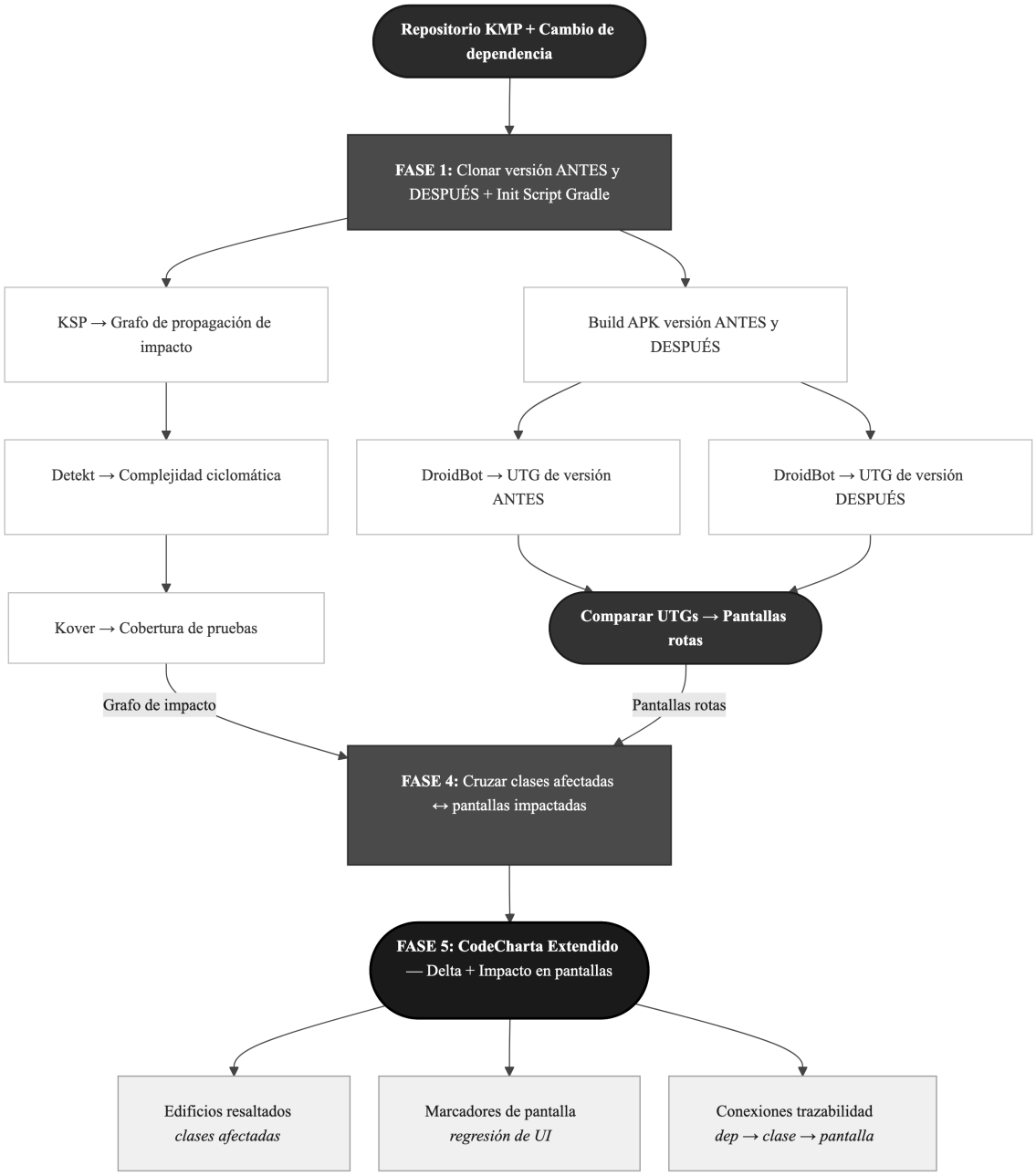

Figura 1: Pipeline de análisis de impacto de cambio de dependencias en proyectos KMP.

3.1. Fase 1: Shadowing y Preparación de Versiones (OE1)

Se clonan dos copias del repositorio KMP: una con la dependencia original (`ANTES`) y otra con la dependencia actualizada (`DESPUES`). En ambas copias se inyecta un Init Script de Gradle Gradle Inc. (2024) que agrega el procesador KSP y Detekt sin modificar los archivos de *build* originales.

3.2. Fase 2: Análisis Estático de Propagación de Impacto (OE1, OE2)

Sobre la versión `DESPUES`, el procesador KSP Google (2024) genera el **grafo de propagación de impacto**: partiendo de la dependencia modificada, identifica todas las clases y funciones que la importan directa o transitivamente, cruzando las fronteras `expect/actual` entre `commonMain`, `androidMain` e `iosMain`. Adicionalmente, Detekt Detekt Team (2024) genera reportes de complejidad ciclomática McCabe (1976) y Kover JetBrains (2024) reportes de cobertura de pruebas para ambas versiones.

3.3. Fase 3: Análisis Dinámico de Regresión de Pantallas (OE3)

Se compilan los APKs de ambas versiones y se despliegan en emuladores Android. DroidBot Li et al. (2017) ejecuta la misma exploración automática (política `dfs_greedy`) sobre cada versión, generando un UTG (*UI Transition Graph*) por versión. Opcionalmente, AutoDroid Wen et al. (2024) ejecuta tareas específicas que ejerciten flujos relacionados con la dependencia modificada. Se comparan los dos UTGs para detectar:

- **Pantallas que fallan** (presentes en UTG-ANTES pero causan crash en DESPUES).
- **Pantallas que cambian** (alcanzables en ambos pero con diferente contenido/estado).
- **Pantallas inalcanzables** (presentes en UTG-ANTES pero no en UTG-DESPUES).

3.4. Fase 4: Consolidación y Cruce Código–Pantalla (OE4)

Un motor de agregación cruza dos fuentes de datos:

- Del análisis estático: el grafo de clases afectadas por el cambio de dependencia.
- Del análisis dinámico: las trazas de métodos de DroidBot, que mapean qué clases se ejecutan en cada pantalla.

El cruce produce una relación **dependencia → clases afectadas → pantallas impactadas**. La Tabla 2 presenta el mapeo de métricas a dimensiones visuales.

Cuadro 2: Mapeo de métricas a dimensiones visuales en CodeCharta extendido.

Dimensión visual	**Métrica**	**Fuente**	**Beneficio**
Tamaño del edificio	LOC	Detekt	Localizar módulos monolíticos
Altura del edificio	Complejidad	Detekt	Zonas de alto riesgo
Color (semáforo)	Cobertura tests	Kover	Zonas sin *unit tests*
Borde resaltado	**Clase afectada**	**KSP**	**Impacto del cambio de dep.**
Marcador de pantalla	**Pantalla rota**	**DroidBot**	**Regresión de UI detectada**
Conexiones	Dep. → clase → pantalla	Cruce	Trazabilidad completa

3.5. Fase 5: Visualización en CodeCharta Extendido (OE4)

Se extiende CodeCharta MaibornWolff GmbH (2024) (*open-source*, TypeScript/Angular) para incorporar:

a) **Capa de impacto**: Los edificios (clases) afectados por el cambio de dependencia se resaltan con un borde (impacto directo vs. transitivo).

b) **Marcadores de pantalla**: Las pantallas detectadas como rotas por DroidBot se representan como marcadores sobre los edificios que las generan, con un ícono de estado (*crash*, cambio, inalcanzable).

c) **Modo Delta enriquecido**: El modo Delta existente se enriquece con la información de impacto, mostrando no solo qué archivos cambiaron sino qué pantallas se afectaron.

3.6. Fase 6: Evaluación de Exactitud del Pipeline (OE5)

La evaluación del pipeline no depende de participantes externos. Se estructura en tres dimensiones cuantitativas, cada una con su propio protocolo de verificación manual experta.

3.6.1. Evaluación del Análisis Estático: ¿Detecta correctamente las clases afectadas?

Para cada escenario de cambio de dependencia se ejecuta el pipeline y se obtiene la lista automática de clases afectadas (directas y transitivas). Luego se realiza una **verificación manual experta** que incluye: revisión del *diff* entre versiones, análisis manual del grafo de dependencias, compilación con *warnings* y errores, e inspección de *stack traces* en caso de fallos. Se compara la lista automática contra la manual usando métricas estándar de *Information Retrieval*:

- **Precision**: $\frac{\text{Clases correctamente detectadas}}{\text{Total de clases reportadas por el pipeline}}$
- **Recall**: $\frac{\text{Clases correctamente detectadas}}{\text{Total de clases realmente afectadas}}$
- **F1-Score**: $2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$

3.6.2. Evaluación del Análisis Dinámico: ¿Detecta correctamente las pantallas rotas?

Para cada cambio de dependencia se ejecuta DroidBot sobre ambas versiones (ANTES y DESPUÉS), se comparan los UTGs, y se obtiene la lista de pantallas con regresión (*crash*, cambio de comportamiento, inalcanzabilidad). Se realiza una **verificación manual controlada**: navegación manual de la app, reproducción de *crashes*, y confirmación de pantallas inaccesibles. Se calculan:

- **Precision y Recall** de pantallas detectadas como rotas.
- **Diferencia de cobertura** de actividades entre ambas versiones.
- **Número de crashes confirmados** versus reportados.

Para mitigar la aleatoriedad inherente a DroidBot, se ejecutan **múltiples corridas** (mínimo 3) por versión, reportando media y desviación estándar de las métricas.

3.6.3. Evaluación de la Visualización: ¿La extensión de CodeCharta conecta las dimensiones?

Se realiza una **evaluación cualitativa estructurada** (sin participantes externos) que demuestra que el modo Delta enriquecido:

- Conecta visualmente la cadena dependencia → clases → pantallas.

- Permite identificar las zonas críticas de impacto más rápido que revisando los reportes XML/JSON individuales de cada herramienta.
- Distingue correctamente impacto directo de transitivo en la visualización.

Se documenta con capturas de pantalla comparativas del modo Delta estándar versus el modo Delta enriquecido sobre los mismos escenarios de cambio de dependencia.

4. Resultados Esperados

1. **Init Script de Gradle funcional** que inyecte KSP y Detekt en cualquier proyecto KMP sin modificar sus archivos de *build*, implementando el principio de *Shadowing*.

2. **Procesador KSP de propagación de impacto** que, dada una dependencia modificada, genere el árbol de clases y funciones afectadas directa y transitivamente, cruzando fronteras `expect/actual`. Exportable en formato JSON.

3. **Comparador de UTGs** que ejecute DroidBot sobre las versiones ANTES y DESPUÉS de un cambio de dependencia, y detecte pantallas con regresión (*crash*, cambio de comportamiento, inalcanzabilidad).

4. **Motor de cruce código–pantalla** que vincule las clases afectadas (estático) con las pantallas impactadas (dinámico), produciendo una relación de trazabilidad completa: dependencia → clases → pantallas.

5. **Extensión de CodeCharta** (*open-source*) con capa de impacto, marcadores de pantalla, y modo Delta enriquecido.

6. **Evaluación cuantitativa** sobre al menos dos escenarios de cambio de dependencia real en proyectos KMP, reportando:

   - Precision, Recall y F1-Score del análisis estático (clases afectadas) contra verificación manual experta.

   - Precision y Recall del análisis dinámico (pantallas rotas) contra verificación manual controlada, con múltiples corridas de DroidBot reportando media y desviación estándar.

   - Evaluación cualitativa estructurada de la visualización con capturas comparativas del modo Delta estándar versus el enriquecido.

La Tabla 3 resume las métricas de evaluación por componente del pipeline.

Cuadro 3: Métricas de evaluación por componente del pipeline.

Componente	**Métrica**	**Verificación**	**Criterio**
Análisis estático (KSP)	Precision, Recall, F1-Score	Revisión manual de *diff*, compilación, *stack traces*	Contra *ground truth* manual
Análisis dinámico (DroidBot)	Precision, Recall de pantallas	Navegación manual, reproducción de *crashes*	Media $\pm\ \sigma$ en múltiples corridas
Visualización (CodeCharta)	Completitud de trazabilidad	Capturas comparativas: Delta estándar vs. enriquecido	Evaluación cualitativa

5. Conclusiones y Trabajos Futuros

Este trabajo propone un pipeline integral y no invasivo para el análisis de impacto de cambios de dependencias en proyectos Kotlin Multiplatform. La contribución central es conectar dos dimensiones que hoy están desconectadas: el impacto en código (análisis estático, grafo de propagación KSP) y el impacto en pantallas (análisis dinámico, comparación de UTGs con DroidBot). La visualización de este impacto en una extensión de CodeCharta permite al desarrollador evaluar el riesgo de un cambio de dependencia antes de aplicarlo.

El principio de *Shadowing*—analizar sin alterar—garantiza adopción industrial con mínima fricción. La metáfora de ciudad, empíricamente validada por Wettel et al. Wettel et al. (2011), proporciona la base visual, y la extensión propuesta (impacto de dependencias + regresión de pantallas) constituye una contribución original a la línea de investigación en visualización de software.

Trabajos futuros:

- Extensión del análisis dinámico a la capa iOS mediante integración con herramientas de *testing* nativas.
- Integración con Dependabot y CI/CD para ejecutar el pipeline automáticamente ante cada alerta de actualización de dependencia.
- Uso de LLMDroid Wang et al. (2025) para generar tareas de exploración específicas a la dependencia modificada, maximizando la cobertura de pantallas relevantes.
- Extensión del procesador KSP para detectar impactos a nivel de API pública (mé-

todos removidos, firmas modificadas) además de dependencias de importación.

- Evaluación empírica con desarrolladores de la industria para medir si la visualización del impacto reduce el tiempo de decisión ante cambios de dependencias.

Referencias

Balogh, G., Gergely, T., Beszédes, Á., and Gyimóthy, T. (2016). Using the city metaphor for visualizing test-related metrics. In *IEEE Working Conference on Software Visualization (VISSOFT)*. IEEE. PDF: `http://www.inf.u-szeged.hu/~beszedes/research/BGB16a.pdf`.

Beriukhov, A. (2025). Kotlin multiplatform scalability challenges on a large project. ProAndroidDev (blog técnico). Reporte de experiencia industrial. `https://proandroiddev.com/kotlin-multiplatform-scalability-challenges-on-a-large-project-b3140e12da9d`.

Biørn-Hansen, A., Majchrzak, T. A., and Grønli, T.-M. (2017). Progressive web apps: The possible web-native unifier for mobile development. In *Proceedings of the 13th International Conference on Web Information Systems and Technologies (WEBIST)*, pages 344–351. PDF: `https://www.scitepress.org/Papers/2017/63537/63537.pdf`.

Custura, S.-O. and Vescan, A. (2025). Code visualization through clothing metaphor: An empirical software comprehension study. In *Procedia Computer Science*, volume 270, pages 1926–1935.

Detekt Team (2024). Detekt: Static code analysis for kotlin. `https://detekt.dev`.

Google (2024). KSP: Kotlin symbol processing API. `https://github.com/google/ksp`.

Gradle Inc. (2024). Initialization scripts and init plugins. `https://docs.gradle.org/current/userguide/init_scripts.html`.

Hao, S., Liu, B., Nath, S., Halfond, W. G. J., and Govindan, R. (2014). PUMA: Programmable UI-automation for large-scale dynamic analysis of mobile apps. In *Proceedings of the 12th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys)*, pages 204–217. ACM.

Janakaraj, M. (2025). Architecting shared logic with kotlin multiplatform mobile: Benefits, challenges, and ecosystem positioning. *European Journal of Computer Science and Information Technology*, 13(44):121–141.

Jeffery, C. (2023). The city metaphor in software visualization: A survey. Technical report, University of Idaho. `https://www.cs.nmt.edu/~jeffery/city-surv.pdf`.

JetBrains (2024). Kotlinx-Kover: Code coverage for kotlin. `https://github.com/Kotlin/kotlinx-kover`.

JetBrains (2025). Kotlin multiplatform development documentation. https://www.jetbrains.com/help/kotlin-multiplatform-dev.

Li, Y., Yang, Z., Guo, Y., and Chen, X. (2017). DroidBot: A lightweight UI-guided test input generator for Android. In *Proceedings of the 39th International Conference on Software Engineering Companion (ICSE-C)*, pages 23–26. IEEE Press. PDF: https://ylimit.github.io/static/files/DroidBot_ICSE2017.pdf.

MaibornWolff GmbH (2024). CodeCharta: Visualize your source code as a city. https://codecharta.com.

Majchrzak, T. A., Biørn-Hansen, A., and Grønli, T.-M. (2017). Comprehensive analysis of innovative cross-platform app development frameworks. In *Proceedings of the 50th Hawaii International Conference on System Sciences (HICSS)*. PDF: https://pdfs.semanticscholar.org/471d/a5675a0eb72247a5f305b776f38ffd181c96.pdf.

McCabe, T. J. (1976). A complexity measure. *IEEE Transactions on Software Engineering*, SE-2(4):308–320. Concepto complementado con: Gill, G. K. y Kemerer, C. F. (1991), Cyclomatic Complexity Density and Software Maintenance Productivity, disponible en: https://sites.pitt.edu/~ckemerer/CK%20research%20papers/CyclomaticComplexityDensity_GillKemerer91.pdf.

Merino, L., Ghafari, M., Anslow, C., and Nierstrasz, O. (2018). A systematic literature review of software visualization evaluation. *Journal of Systems and Software*, 144:165–180. Reemplaza a Koschke (2003). PDF no disponible; contenido verificado vía DOI.

Moreno-Lumbreras, D., González-Barahona, J. M., Robles, G., and Cosentino, V. (2024). The influence of the city metaphor and its derivates in software visualization. *Journal of Systems and Software*, 210:111985.

Novendra, R. D. and Sunindyo, W. D. (2024). Emerging trends in code quality: Introducing kotlin-specific bad smell detection tool for android apps. *IEEE Access*, 12:63894–63907. Open Access: https://ieeexplore.ieee.org/document/10520915.

Pan, M., Huang, A., Wang, G., Zhang, T., and Li, X. (2020). Reinforcement learning based curiosity-driven testing of android applications. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM.

Suri, B., Taneja, S., and Bhanot, I. (2023). Cross-platform empirical analysis of mobile application development frameworks: Kotlin, react native and flutter. In *Proceedings of the 2023 International Conference on Electrical, Electronics and Computing Technologies*.

Wang, Y., Liu, J., Chen, J., and Jiang, H. (2025). LLMDroid: Enhancing automated mobile app GUI testing coverage with large language model guidance. In *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE)*. ACM. https://dl.acm.org/doi/10.1145/3715763.

Wen, H., Li, Y., Liu, G., Zhao, S., Yu, T., Li, T. J.-J., Jiang, S., Liu, Y., Zhang, Y., and Liu, Y. (2024). AutoDroid: LLM-powered task automation in Android. arXiv preprint arXiv:2308.15272. https://arxiv.org/abs/2308.15272.

Wettel, R., Lanza, M., and Robbes, R. (2011). Software systems as cities: A controlled experiment. In *Proceedings of the 33rd International Conference on Software Engineering (ICSE)*, pages 551–560. ACM. PDF: https://www.researchgate.net/publication/221555737.